Primerjava centraliziranega in distribuiranega več-agentnega iskanja poti

Andraž Šošterič
andraz.sosteric@student.um.si
Faculty of Electrical
Engineering and Computer Science,
University of Maribor
Koroška cesta 46
SI-2000 Maribor, Slovenia

David Lipavec
david.lipavec@student.um.si
Faculty of Electrical
Engineering and Computer Science,
University of Maribor
Koroška cesta 46
SI-2000 Maribor, Slovenia

Nejc Fras
nejc.fras@student.um.si
Faculty of Electrical
Engineering and Computer Science,
University of Maribor
Koroška cesta 46
SI-2000 Maribor, Slovenia

Tadej Teršek
tadej.tersek@student.um.si
Faculty of Electrical
Engineering and Computer Science,
University of Maribor
Koroška cesta 46
SI-2000 Maribor, Slovenia

KLJUČNE BESEDE

centralizirano načrtovanje poti, distribuirano načrtovanje poti, usklajevanje agentov, izogibanje trkom, optimizacija poti

1 UVOD

V tem članku obravnavamo problem več-agentnega iskanja poti (Multi-Agent Path Finding / MAPF), ki se v praksi pojavlja v številnih domenah, kot so robotizirana skladišča, avtonomna vozila, inteligentni transportni sistemi in drugi logistični sistemi. Gre za temeljni problem koordinacije več avtonomnih enot v skupnem prostoru, kjer mora vsak agent doseči svoj cilj brez trkov z drugimi agenti ali ovirami.

Cilj problema je načrtovati poti za množico agentov v diskretnem prostoru tako, da se izognemo medsebojnim konfliktom in hkrati optimiziramo določene metrike, kot sta skupni čas izvajanja (ang. *makespan*) ali vsota dolžin poti (ang. *sum of costs*). [9]

Repozitorij z izvorno kodo projekta je na voljo na naslovu: https://github.com/Asos75/MultiAgent.

Kot osnovo za formalno definicijo problema in uporabljene metrike uporabljamo delo *Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks* [9]. Poudarjamo, da omenjeni članek ne predstavlja konkretnega algoritma, temveč služi kot referenčni okvir za definicije problema MAPF, njegove različice ter standardne metrike, ki jih uporabljajo v nadaljevanju omenjeni članki in katere bomo uporabili tudi mi.

Za analizo bomo uporabili več sodobnih znanstvenih člankov, ki obravnavajo različne pristope k reševanju problema MAPF. Izbrane članke so identificirali posamezni člani skupine:

- *LaCAM: Search-Based Algorithm for Quick Multi-Agent Pathfinding* [6] (Šošterič),
- *Priority-Based Search for Multi-Agent Path Finding* [5] (Lipavec),
- *Follower: Learning to Solve Decentralized Lifelong Multi-Agent Pathfinding* [1] (Teršek),
- *MATS-LP: Learning Decentralized Multi-Agent Pathfinding with Transformer-Based Policies* [8] (Fras).

V nadaljevanju bomo primerjali centralizirane metode, katerih predstavnika sta LaCAM in PBS, ki načrtujejo poti za vse agente z uporabo globalnega pogleda na problem, ter decentralizirane pristope, kot sta Follower in MATS-LP, kjer posamezni agenti sprejemajo odločitve lokalno brez centralnega nadzora in imajo pregled le nad svojo okolico.

Na ta način bomo analizirali ključne razlike med obema paradigmama z vidika učinkovitosti, skalabilnosti in kakovosti rešitev. Centralizirani pristopi praviloma zagotavljajo bolj optimalne rešitve, vendar slabše skalirajo z naraščajočim številom agentov, medtem ko decentralizirani pristopi omogočajo boljšo razširljivost, vendar pogosto na račun kakovosti rešitev.

Kot glavni referenčni algoritem izberemo delo *LaCAM: Search-Based Algorithm for Quick Multi-Agent Pathfinding* [6], ki predstavlja sodoben in učinkovit centraliziran pristop k reševanju problema MAPF. Izbrani članek služi kot referenčna osnova za našo analizo, saj ponuja dobro ravnovesje med učinkovitostjo in skalabilnostjo ter omogoča relativno enostavno reprodukcijo eksperimentov.

LaCAM bomo uporabili kot predstavnika centraliziranih metod, s katerim bomo primerjali decentralizirane pristope, kot sta Follower in MATS-LP, ter tako sistematično analizirali razlike med globalnim in lokalnim načrtovanjem poti.

1.1 Uvedba lokalnih vodij

Glavni prispevek tega dela je predlagana nadgradnja obstoječih pristopov, ki uvaja koncept *lokalnih vodij*. Gre za hibridni pristop med centraliziranim in decentraliziranim načrtovanjem, kjer je množica agentov razdeljena na lokalne skupine.

Vsako skupino nadzira izbrani agent, lokalni vodja, ki koordinira gibanje agentov v svoji neposredni okolici in rešuje konflikte na lokalni ravni. Vodja nima globalnega pogleda, temveč ima pogled

le nad okolico, ki jo vidijo njegovi agenti. S tem zmanjšamo potrebo po globalni koordinaciji, hkrati pa ohranimo določeno stopnjo usklajenosti med agenti.

S predlaganim pristopom želimo združiti prednosti obeh paradigem: visoko kakovost rešitev, značilno za centralizirane metode, ter boljšo skalabilnost in robustnost, značilno za decentralizirane pristope. V eksperimentalnem delu bomo preverili, ali tak hibridni pristop dejansko izboljša učinkovitost reševanja problema MAPF v različnih scenarijih.

2 DEFINICIJA PROBLEMA

Stern et al. [9] definirajo problem več-agentnega iskanja poti (Multi-Agent Path Finding - MAPF) kot trojico

$$\langle G, s, t \rangle,$$

kjer je $G = (V, E)$ neusmerjen graf, pri čemer V predstavlja množico vozlišč, E pa množico povezav.

Podana je množica agentov $A = \{1, 2, \ldots, k\}$. Funkcija

$$s : \{1, \ldots, k\} \to V$$

preslika vsakega agenta v njegovo začetno vozlišče, funkcija

$$t : \{1, \ldots, k\} \to V$$

pa v njegovo ciljno vozlišče.

Naj bo $\pi_i[t] \in V$ položaj agenta i v diskretnem času $t \in \mathbb{N}_{\geq 0}$. Pri vsakem koraku t se agent i lahko premakne v sosednje vozlišče ali ostane na trenutnem, tj.

$$\pi_i[t+1] \in \text{neigh}(\pi_i[t]) \cup \{\pi_i[t]\},$$

kjer je $\text{neigh}(v)$ množica vozlišč, sosednjih vozlišču $v \in V$. [6]

Po Okamuri [6] se mora agent izogibati naslednjim vrstam konfliktov:

- **Vozliščni konflikt:** Do vozliščnega konflikta (ang. vertex conflict) pride, če nameravata π_i in π_j zasedeti isto vozlišče v istem trenutku, torej obstaja časovni korak t, da velja $\pi_i[t] = \pi_j[t]$. [9]
- **Zamenjalni konflikt:** Do zamenjalnega konflikta (ang. swap conflict) pride, če nameravata π_i in π_j zamenjati svoji poziciji v istem trenutku, torej obstaja časovni korak t, da velja $\pi_i[t] = \pi_j[t+1] \wedge \pi_i[t+1] = \pi_j[t]$. [9]

Rešitev MAPF za instance, omenjene zgoraj, je množica brezkonfliktnih poti $\{\pi_1, \ldots, \pi_k\}$ takih, da obstaja časovni korak $T \in \mathbb{N}_{\geq 0}$, kjer agent doseže cilj. [6]

3 SORODNA DELA

V tem razdelku izpostavimo ključna dela s področja več-agentnega iskanja poti, ki dopolnjujejo in razširjajo pogled iz uvoda.

3.1 Centralizirani pristopi

EECBS [3] uvaja suboptimalno različico CBS z uporabo Explicit Estimation Search in dosega boljšo skalabilnost pri večjem številu agentov. Predstavlja pomembno referenco za kompromis med optimalnostjo in učinkovitostjo, ki je ključen pri primerjavi z algoritmi, kot sta LaCAM in PBS.

RHCR [4] razdeli lifelong MAPF na zaporedje krajših problemov z omejenim časovnim horizontom, kar omogoča učinkovito ponovno načrtovanje. Gre za standarden centraliziran pristop za lifelong scenarije, primerljive z našimi eksperimentalnimi nastavitvami.

3.2 Decentralizirani pristopi

Follower [1] je decentraliziran pristop k reševanju problema lifelong MAPF, ki temelji na kombinaciji hevristično načrtovane poti in naučene politike sledenja. Sistem je sestavljen iz dveh komplementarnih modulov.

Hevristični načrtovalec poti za vsakega agenta neodvisno zgradi pot od trenutne lokacije do cilja z algoritmom A*. Pri tem upošteva le statične ovire, ne pa tudi ostalih agentov, saj se izogibanje trkom prepusti drugemu modulu. Ključna lastnost načrtovalca je, da ne išče najkrajše poti, temveč enakomerno razpršene poti, s čimer zmanjša zasičenost na ozkih mestih karte (hodniki, vrata). To doseže z dvonivojskimi stroški prehoda: statični stroški temeljijo na topologiji karte in penalizirajo pogosto uporabljena vozlišča, dinamični stroški pa se tekom epizode posodabljajo glede na agentove lastne opazovalne izkušnje.

Modul za sledenje poti implementira naučeno politiko, ki agenta vodi vzdolž konstruirane poti in hkrati zagotavlja izogibanje trkom z ostalimi agenti. Politiko aproksimira globoka nevronska mreža, ki kot vhod sprejme lokalno opazovanje agenta v obliki 2D tenzorja dimenzije 2×m×m. Tenzor kodira lokacije statičnih ovir, trenutno pot in položaje ostalih agentov. Arhitektura mreže sestoji iz prostorskega koderja, ki mu sledita dve veji – akterska in kritična – obe z rekurenčnimi bloki GRU za modeliranje zgodovine opazovanj.

Sistem se uči z algoritmom PPO, kjer agent prejme nagrado za dosego naslednje točke na poti. Kadar se agent preveč oddalji od poti ali se ji preveč približa, se hevristični načrtovalec ponovno aktivira. Med decentraliziranim sklepanjem vsak agent uporablja kopijo naučenih uteži, kar omogoča enostavno skaliranje na poljubno število agentov brez potrebe po centralnem koordinatorju ali ekspertnih podatkih za učenje.

PRIMAL2 [2] združuje podkrepitveno učenje in posnemanje za delovanje na podlagi lokalnih opazovanj brez centralnega koordinatorja. Predstavlja zgodnji in vpliven pristop, ki osvetljuje razvoj sodobnih decentraliziranih metod.

SCRIMP [10] uvaja komunikacijo med agenti z uporabo transformerjev, kar omogoča koordinacijo kljub omejenemu vidnemu polju. Predstavlja alternativen pristop k usklajevanju agentov v primerjavi z metodami brez eksplicitne komunikacije.

3.3 Primerjalne platforme

POGEMA [7] ponuja standardizirano platformo za evalvacijo MAPF algoritmov, vključno z generatorjem instanc in enotnimi metrikami. Omogoča pošteno in ponovljivo primerjavo različnih pristopov, kar je ključno za našo eksperimentalno analizo.

4 PREGLED OVREDNOTENJA

Za primerjavo izbranih algoritmov bomo uporabili standardne metrike s področja MAPF, ki so uveljavljene v obstoječi literaturi [7, 9]. Vrednotenje bo potekalo na dveh standardnih zbirkah testnih instanc: MovingAI MAPF benchmark [9] in POGEMA benchmark [7].

4.1 Metrike

Sum-of-Costs (SoC) je vsota časovnih korakov, ki jih vsi agenti porabijo za dosego svojih ciljnih vozlišč [9]:

$$\text{SoC} = \sum_{i=1}^{k} |\pi_i|,$$

kjer je $|\pi_i|$ dolžina poti agenta i. Nižja vrednost pomeni učinkovitejšo rešitev.

SoC je ena izmed najpogosteje uporabljenih metrik v MAPF literaturi [9], kar omogoča neposredno primerjavo z obstoječimi deli. Metrika meri skupni strošek kot vsoto dolžin poti vseh agentov in s tem neposredno zajame celostno učinkovitost sistema.

Metrika je še posebej primerna za one-shot scenarije, saj minimizacija SoC ustreza cilju minimizacije skupne porabe časa oziroma virov. Hkrati kaznuje neučinkovite poti posameznih agentov, kar spodbuja iskanje globalno učinkovitih rešitev. SoC bomo zato uporabili za vrednotenje algoritmov LaCAM in PBS na zbirki MovingAI.

Makespan je čas, ki ga porabi najdaljša pot med vsemi agenti, torej maksimum nad vsemi $|\pi_i|$ [9]:

$$\text{makespan} = \max_{i \in \{1,\ldots,k\}} |\pi_i|.$$

Metrika odraža skupno trajanje reševanja instance in je primerna za scenarije, kjer je ključen vzporedni zaključek vseh agentov. Makespan je uveljavljena metrika v MAPF literaturi [9] in je še posebej relevantna v praktičnih aplikacijah, kjer je ključen vzporedni zaključek vseh agentov.

Makespan je pri primerjavi centraliziranih algoritmov nujna dopolnitev k SoC, saj sama vsota stroškov namreč ne razkrije, ali posamezen agent morda bistveno zavlačuje celotno skupino. Z vključitvijo obeh metrik tako dobimo celovitejšo in bolj zanesljivo oceno učinkovitosti algoritma v realnih pogojih.

Stopnja uspeha (Success Rate) meri delež instanc, pri katerih vsi agenti uspešno dosežejo svoje cilje znotraj predvidenega časovnega horizonta [7].

Stopnja uspeha je primarna metrika platforme POGEMA in skupaj s pretokom (ang. throughput) tvori osnovo za vrednotenje na tej platformi [7]. Zasnovana je posebej za vrednotenje decentraliziranih metod v delno opazljivih okoljih. Metrika je za naš eksperiment ključna, saj za razliko od SoC in makespana meri tudi robustnost algoritma oziroma njegovo sposobnost, da instanco v celoti reši kljub omejenim lokalnim opazovanjem.

Čas načrtovanja merimo kot skupni čas izvajanja algoritma na posamezni instanci. Ta metrika je še posebej pomembna pri primerjavi med kategorijama, saj centralizirani načrtovalec tipično porabi več časa za globalno načrtovanje, kot decentralizirani agenti.

Čas načrtovanja je v praktičnih aplikacijah enako pomemben kot kakovost rešitve, saj algoritem z optimalno potjo, a dolgim časom načrtovanja, v realnih sistemih ni uporaben. Ta metrika nam tako omogoča primerjavo med pristopoma z vidika razširljivosti in praktične uporabnosti.

4.2 Protokol primerjave

Primerjavo bomo izvedli na dveh ravneh:

(1) **Znotraj kategorije:** LaCAM in PBS primerjamo na MovingAI zbirki po metrikah SoC, makespan in čas načrtovanja. Follower in MATS-LP primerjamo na POGEMA zbirki po metrikah throughput in stopnja uspeha.
(2) **Med kategorijama na skupni zbirki:** Obe katergoriji bomo primerjali med seboj na skupni zbirki, kjer bomo primerjali prej navedene metrike.

5 NAČRT REŠITVE

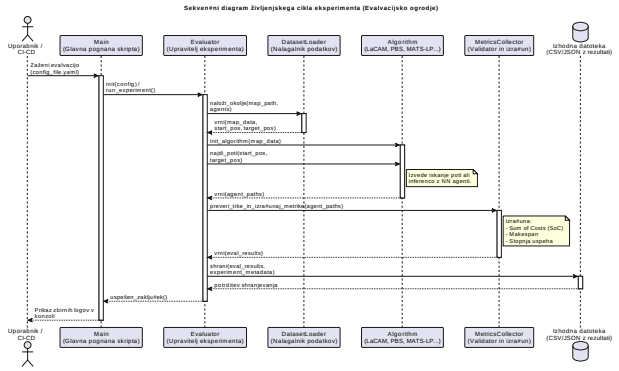
V tem poglavju so navedena opravila, potrebna za implementacijo projekta, ki so razdeljena v štiri razvojne iteracije (sprinte). Predstavljena sta tudi časovni razpored v obliki Ganttovega (Table 1) diagrama, ter UML diagram predlagane rešitve (Figure 1).

- **Sprint 1:**
  - Ustvarjanje repozitorija Git za nalaganje, verzioniranje in usklajevanje programske kode.
  - Pisanje osnutka poglavja *Uvod* (predstavitev problema MAPF in motivacije).
  - Pisanje osnutka poglavja *Sorodna dela* (pregled algoritmov LaCAM, PBS, Follower in MATS-LP).
  - Pridobivanje in prenos izvorne kode izbranih centraliziranih in decentraliziranih algoritmov.
  - Osnovna analiza strukture implementacij.
  - Priprava eksperimentalnega okolja (namestitev POGEMA, nastavitev C++ in Python okolja).
- **Sprint 2:**
  - Dokončanje poglavij *Uvod* in *Sorodna dela*.
  - Pisanje poglavja *Metodologija* (definicija metrik: *SoC, makespan, uspešnost, čas*).
  - Začetek poglavja *Eksperiment in analiza rezultatov* (opis eksperimentalnega protokola, instanc in parametrov).
  - Reprodukcija rezultatov iz izbranih člankov na manjših instancah (validacija implementacij).
  - Implementacija lastne ideje:
    * zasnova enotnega evalvacijskega ogrodja (*pipeline*),
    * definicija skupnega vmesnika za integracijo algoritmov.
  - Implementacija eksperimentalnega ogrodja in osnovnih skript za zagon eksperimentov.
- **Sprint 3:**
  - Dokončanje poglavja *Metodologija*.
  - Nadaljevanje poglavja *Eksperiment in analiza rezultatov*.
  - Nadaljnja implementacija lastne ideje:
    * razdelitev agentov v lokalne skupine,
    * določitev lokalnih vodij,
    * implementacija lokalne koordinacije in reševanja konfliktov.
  - Implementacija metrik znotraj ogrodja (SoC, makespan, uspešnost, čas izvajanja).
  - Integracija vseh algoritmov v skupno ogrodje.
  - Zagon lastnega eksperimenta:
    * na lastni izboljšavi,

    - na izbranih instancah iz zbirk MovingAI in POGEMA,
    - pri različnih gostotah agentov in velikostih map.
- **Sprint 4:**
  - Dokončanje poglavja *Eksperimenti in analiza rezultatov*.
  - Pisanje poglavja *Zaključek*.
  - Pisanje poglavja *Povzetek*.
  - Podrobna analiza rezultatov na podlagi definiranih metrik.
  - Primerjava centraliziranih, decentraliziranih in hibridnega pristopa.
  - Identifikacija prednosti, slabosti in omejitev posameznih pristopov.
  - Izdelava vizualizacij (grafi in tabele).
  - Končna revizija članka in priprava za oddajo.

Tabela 1: Ganttov diagram razvoja po sprintih.

Naloga	**Tedni**							
	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Sprint 1								
Ustvarjanje Git repozitorija	■							
Pisanje poglavja *Uvod*	■	■						
Pisanje poglavja *Sorodna dela*	■	■						
Pridobivanje programske kode	■							
Priprava eksperimentalnega okolja	■							
Sprint 2								
Dokončanje *Uvod* in *Sorodna dela*		■	■					
Pisanje *Metodologija*			■	■				
Opis eksperimenta			■	■				
Reprodukcija rezultatov				■				
Razvoj evalvacijskega ogrodja				■	■			
Sprint 3								
Dokončanje *Metodologija*					■			
Implementacija ideje in metrik					■	■		
Integracija algoritmov					■			
Zagon eksperimentov (MovingAI/POGEMA)						■	■	
Sprint 4								
Dokončanje *Eksperimenti in analiza*							■	■
Analiza rezultatov							■	■
Pisanje *Zaključek*								■
Pisanje *Povzetek*								■
Revizija in oddaja								■



Slika 1: UML diagram (diagram primera uporabe / diagram razredov / sekvenčni diagram) predlagane rešitve.

LITERATURA

[1] Anton Andreychuk, Konstantin Yakovlev, Alexey Skrynnik, and Aleksandr Panov. 2024. Follower: Learning to Solve Decentralized Lifelong Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press.

[2] Mehul Damani, Zhiyao Luo, Emerson Wenzel, and Guillaume Sartoretti. 2021. PRIMAL$_2$: Pathfinding via Reinforcement and Imitation Multi-Agent Learning — Lifelong. *IEEE Robotics and Automation Letters* 6, 2 (2021), 2666–2673. https://doi.org/10.1109/LRA.2021.3062803

[3] Jiaoyang Li, Wheeler Ruml, and Sven Koenig. 2021. EECBS: Bounded-Suboptimal Search for Multi-Agent Path Finding. In *Proceedings of 35th AAAI Conference on Artificial Intelligence (AAAI '21)*. 12353 – 12362.

[4] Jiaoyang Li, Andrew Tinka, Scott Kiesel, Joseph W. Durham, T. K. Satish Kumar, and Sven Koenig. 2021. Lifelong Multi-Agent Path Finding in Large-Scale Warehouses. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 11272–11281. https://doi.org/10.1609/aaai.v35i13.17344

[5] Hang Ma, Daniel Harabor, Peter J. Stuckey, Jiaoyang Li, and Sven Koenig. 2019. Searching with Consistent Prioritization for Multi-Agent Path Finding. *Proceedings of the AAAI Conference on Artificial Intelligence* 33, 01 (Jul. 2019), 7643–7650. https://doi.org/10.1609/aaai.v33i01.33017643

[6] Keisuke Okumura. 2023. LaCAM: Search-Based Algorithm for Quick Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. AAAI Press, 11655–11662. https://doi.org/10.1609/aaai.v37i10.26377

[7] Alexey Skrynnik, Anton Andreychuk, Anatolii Borzilov, Alexander Chernyavskiy, Konstantin Yakovlev, and Aleksandr Panov. 2025. POGEMA: A Benchmark Platform for Cooperative Multi-Agent Pathfinding. In *The Thirteenth International Conference on Learning Representations*.

[8] Alexey Skrynnik, Anton Andreychuk, Konstantin Yakovlev, and Aleksandr Panov. 2024. Learning Decentralized Policies for Multi-Agent Pathfinding with Transformers. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

[9] Roni Stern, Nathan Sturtevant, Ariel Felner, Sven Koenig, Hang Ma, Thayne Walker, Jiaoyang Li, Dor Atzmon, Liron Cohen, T. K. Satish Kumar, Eli Boyarski, and Roman Bartak. 2019. Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks. In *Proceedings of the 12th Annual Symposium on Combinatorial Search (SoCS)*.

[10] Yutong Wang, Bairan Xiang, Shao Huang, and Guillaume Sartoretti. 2023. SCRIMP: Scalable Communication for Reinforcement- and Imitation-Learning-Based Multi-Agent Pathfinding. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. https://doi.org/10.1109/IROS55552.2023.10341379